Chapter 1: Exponential Smoothing

Honours Time Series Module

September 2026

---

1 Introduction

You may have encountered exponential smoothing in undergraduate courses. We will revisit this topic here, firstly because it is widely used in practice, fairly successfully, and secondly, because exponential smoothing has links to state-space models (Hyndman et al., 2002).

Exponential smoothing is mainly a forecasting tool, and in this form not based on a statistical model (although we will see later how state-space models can be used for exponential smoothing). The main idea is that forecasts are calculated as exponentially weighted averages of past observations using updating equations.

A cautionary note on forecasting: Forecasting is like driving a car while only looking backwards!

We will look at the following topics:

- Simple Exponential Smoothing
- Double Exponential Smoothing (Holt's method): incorporates trend
- Triple Exponential Smoothing (Holt-Winters): incorporates trend and seasonality

- ETS models: error, trend, season

The exponential smoothing approach breaks time series data up into components: error, level, trend and seasonal. The observed data is constructed from these components in an additive, damped or multiplicative manner.

These notes are based on various sources: Chapter 8 of Hyndman & Athanasopoulos (https://otexts.com/fpp3/expsmooth.html), and their R packages forecast, fable, fpp3, fpp2. However, the emphasis here is on a deeper understanding of the methods used. Other sources used are Chatfield 2004 Chapter 5 (Forecasting), Shumway & Stoffer (2011), Metcalfe & Cowpertwait (2009, Section 3.4).

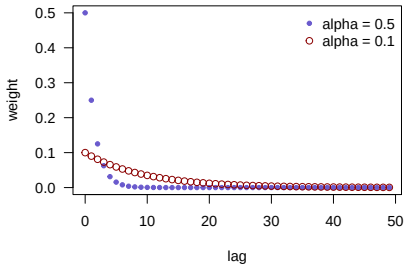


Figure 1: Weights in Simple Exponential Smoothing

2 Simple/Single Exponential Smoothing (SES)

With simple exponential smoothing, forecasts are constructed by using a weighted average of observations, with recent observations contributing more and weights for observations further back into the past decreasing exponentially.

$$\begin{aligned}\hat{y}_{t+1|t} &= \alpha \sum_{j=0}^{t-1} (1-\alpha)^j y_{t-j}, \qquad (0 < \alpha < 1) \\ &= \alpha y_t + \alpha(1-\alpha) y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \ldots\end{aligned}$$

α is called the smoothing parameter.

A typical application is forecasting sales of a well-established product in a stable market.

The prediction for time point $t+1$ is based on a weighted average of the observations (history) up to time t. The weights decrease exponentially. For this reason simple exponential smoothing is also known as exponentially weighted moving average (EWMA). The prediction shown above is a one-step-ahead forecast.

Notation: I will try and stick to this notation throughout these notes: hats for predictions, $\hat{y}_{t+1|t}$ denotes the prediction for time point $t+1$ based on observations (history) up to time t.

Two alternative approaches for obtaining a one-step-ahead forecast would be a naive forecast and the average method. Both can be seen as special cases of simple exponential smoothing.

Simple exponential smoothing results in *flat forecasts*:

$$\hat{y}_{t+k|t} = \hat{y}_{t+1|t}$$

There is no new information after time t, so all forecasts use this same weighted average of the observed data points.

For t large, the weights $(1-\alpha)^j\alpha$ sum approximately to one.

We can re-write the one-step-ahead forecast as follows.

$$\begin{aligned}\hat{y}_{t+1|t} &= \alpha\sum_{j=0}^{t-1}(1-\alpha)^j y_{t-j}, \qquad (0 \le \alpha < 1)\\ &= \alpha y_t + \alpha(1-\alpha)y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \dots\\ &= \alpha y_t + (1-\alpha)\hat{y}_{t|t-1}\end{aligned}$$

In the last line, the forecast has been written as a weighted average of the last observation (at time t) and the forecast for time t based on observations up to time $t-1$.

We can also write this in terms of updating equations:

$$\begin{aligned}\text{Forecast equation} \qquad & \hat{y}_{t+h|t} = \ell_t\\ \text{Smoothing equation} \qquad & \ell_t = \alpha y_t + (1-\alpha)\ell_{t-1},\end{aligned}$$

ℓ: level. This can be understood as a local or current mean.

If we think of data points as noisy observations from an underlying distribution with mean / level ℓ_t, the level can be understood as the best estimate of the level at time t based on the current observations and history. The actual observation at time t is not necessarily the best estimate, because it is noisy and past observations can be used to improve this estimate.

Naive forecast: $\hat{y}_{t+h|t} = y_t$

Average method: $\hat{y}_{t+h|t} = \frac{1}{t}\sum_{i=1}^{t} y_i$

Actually, the weights form a geometric series, they decrease by a constant ratio:
$c_i = \alpha(1-\alpha)^i, \qquad i = 0, 1, \dots$
ensures that the weights sum to 1 (Figure 1).

Sum of geometric series:
$a + ar + ar^2 + \dots + ar^{n-1} = \sum_{k=0}^{n-1} ar^k = a\left(\frac{1-r^n}{1-r}\right)$

The smoothing parameter is used to decide how much weight to give to the observation vs history. The second equation is called 'smoothing equation' because weighted averages will smooth the time series. We first update the level and then use that to forecast.

To *start* the recursive updating equations there are two approaches:

1) Start with: $\ell_1 = y_1; \hat{y}_{2|1} = y_1$ or,
2) set $\hat{y}_1 = \ell_0$, where ℓ_0 is estimated from the data (as an extra parameter). In most cases, we also need to estimate α from the data.

In time series analyses we also use the one-step-ahead forecasts as **fitted values**.

2.1 Estimating smoothing parameter and initial value

α and ℓ_0 can be estimated by minimizing the error sum of squares (SSE).

Residuals are $e_t = y_t - \hat{y}_{t|t-1}$ (observed - fitted).

$$SSE = \sum_{t=1}^{T} (y_t - \hat{y}_{t|t-1})^2$$

Values of α between 0.1 and 0.3 are commonly used if not estimated. Such values result in forecasts which depend on a large number of past observations, and will react slowly to changes in the mean.

For simple exponential smoothing, once we have α and ℓ_0 we don't need to rerun the algorithm, but can update as observations come in. A disadvantage of SES is that there is no error distribution or any stochastic model involved. This makes it more difficult to specify the uncertainty of estimates and forecasts.

What is the difference between level, forecast, fitted value?

2.2 Example 1

To illustrate, here is a small simulated example. There are 20 observations and no systematic trend (Figure 2). The model used for simulation is

$$\ell_t = \ell_{t-1} + \alpha e_t, \quad y_t = \ell_{t-1} + e_t, \quad e_t \sim N(0, \sigma^2)$$
$$\ell_0 = 3, \alpha = 0.4, \sigma = 0.5$$

The above values are simulated according to a simple state-space model. We will look at state-space models later. For now they assume an **underlying, unobserved state**. Observations are generated conditional on the current state. Here, the underlying state would be ℓ_t, and observations are generated as $y_t = \ell_{t-1} + \epsilon_t$. This is similar to the updating equations for simple exponential smoothing, except with two error sources (observation error and variation in state / level).

```
set.seed(24)
N <- 100
y <- numeric(N)
level <- numeric(N)
err <- numeric(N)
sigma <- 0.5
alpha = 0.4
l0 = 3

err[1] <- rnorm(1, mean = 0, sd = sigma)
level[1] <- l0 + alpha * err[1]
y[1] <- l0 + err[1]

for (i in 2:N) {
  err[i] <- rnorm(1, mean = 0, sd = sigma)
  level[i] <- level[i-1] + alpha * err[i]
  y[i] <- level[i-1] + err[i]
}

plot.ts(y, las = 1, xlab = "time",
        col = "forestgreen")
```

Using simple exponential smoothing, α and ℓ_0 can be estimated by minimizing the error sum of squares. We will use only the first 50 observations to estimate the parameters and then the

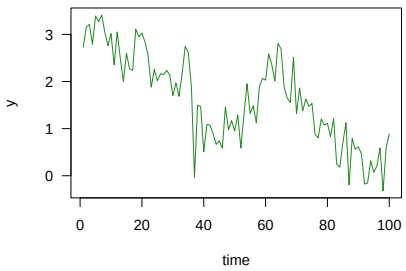


Figure 2: Simulated time series with no systematic trend. These data were simulated from the state-space model $\ell_t = \ell_{t-1} + \alpha e_t, \quad y_t = \ell_t + e_t, \quad e_t \sim N(0, \sigma^2), \quad \ell_0 = 3, \alpha = 0.4, \sigma = 0.5$.

rest of the data to check how well we can predict. $\hat{\ell}_0 = 2.96, \hat{\alpha} = 0.44, \hat{\sigma} = 0.495$.

```
SSE <- function(pars, y) {
  level0 <- pars[1]
  alpha <- pars[2]
  n <- length(y)

  yhat <- numeric(n)
  level <- numeric(n)

  yhat[1] <- level0

  for (i in 1:(n)) {
    level[i] <- alpha * y[i] + (1- alpha) * yhat[i]
    yhat[i+1] <- level[i]
  }

  SSE <- sum((y - yhat[1:n])^2)
  return(SSE)
}

inits <- c(y[1], 0.5)
(ses.est <- optim(par = inits, fn = SSE, y = y[1:50]))
```

```
$par
[1] 2.9600935 0.4362977

$value
[1] 11.77175

$counts
function gradient
      45       NA

$convergence
[1] 0

$message
NULL
```

How is $\hat{\sigma} = 0.495$ calculated? Hint: For this particular example $SSE = 11.77175$.

```
library(forecast)

ses.out <- ses(y[1:50])
ses.out$model
```

```
Simple exponential smoothing

Call:
ses(y = y[1:50])

  Smoothing parameters:
    alpha = 0.4363

  Initial states:
    l = 2.9598

  sigma:  0.4952

     AIC     AICc      BIC
129.2851 129.8069 135.0212
```

Once we have obtained estimates for α and ℓ_0, we can add level estimates, fitted values, and forecasts for all other time points (Figure 3). Note how ℓ_t (blue line) is a weighted average of observation (black line) and forecast (red line). The blue line is smoother than the observations, and will mostly give a better idea of the true level (grey line) than the forecasts. The red line (forecasts, fitted values) lags one step behind the blue line (smoothed values).

A close-up illustrates the updating steps (Figure 4): ℓ_9 is used to forecast at time 10. When observation y_{10} comes in, this forecast gets updated to ℓ_{10}, which in turn is used to forecast for time 11.

2.3 When is SES appropriate?

If trend and seasonality are present, we can do a lot better than simple exponential smoothing. However, it is useful to consider what kind of process we are trying to capture with simple exponential smoothing, and hence for which kind of process simple exponential smoothing might work well.

Shumway & Stoffer (2011, Section 3.7 Example 3.37) show that an ARIMA(0, 1, 1) model leads to simple exponential smoothing

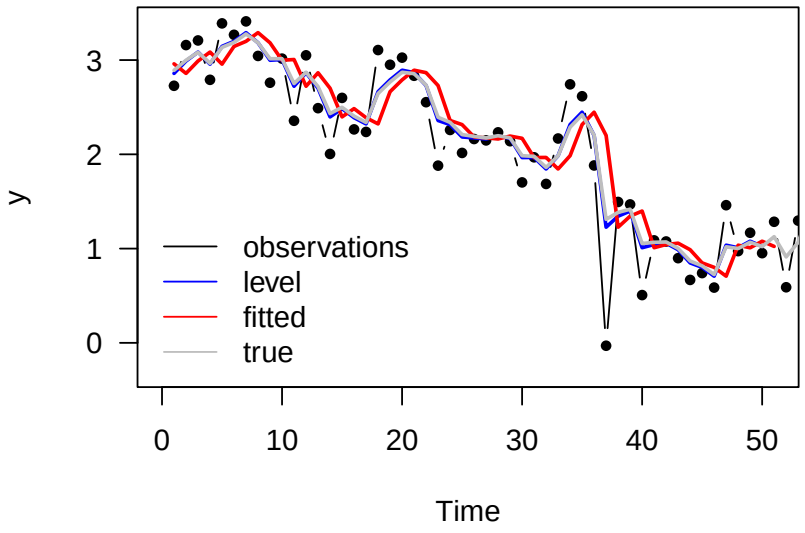


Figure 3: Small data set (black) with one-step-ahead forecasts (red), smoothed level (blue) and true level (grey).

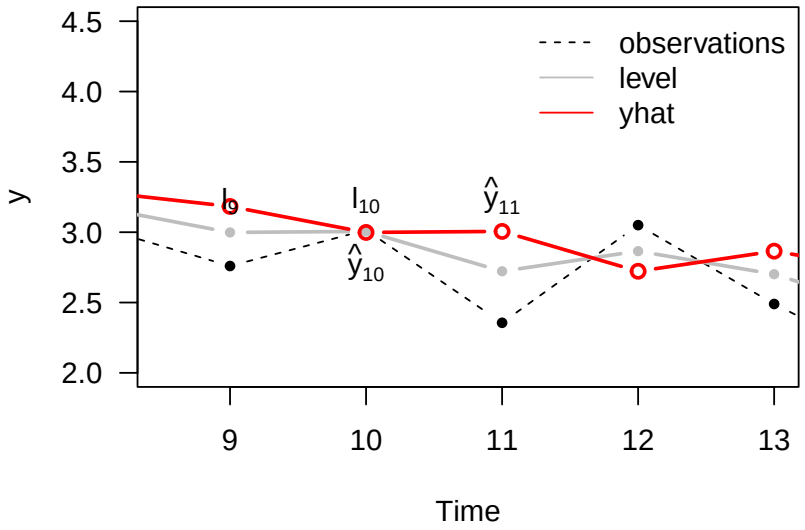


Figure 4: Close-up of small data set with observations (black), estimated level (grey), and one-step-ahead forecasts (red).

forecasts (see Exercises). They show that for the ARIMA(0, 1, 1) model

$$y_t = y_{t-1} + w_t - \lambda w_{t-1}$$

the forecast is

$$\hat{y}_{n+1} = (1 - \lambda) y_t + \lambda \hat{y}_t.$$

This forecast has the same form as that of simple exponential smoothing with smoothing parameter $1 - \lambda$.

In the following example 100 values are simulated from an ARIMA(0, 1, 1) model, and then simple exponential smoothing (SES) is used for forecasting (function `ses` from the `forecast` R package).

```
library(forecast)
set.seed(666)
x <- arima.sim(list(order = c(0, 1, 1), ma = -0.8), n = 100)
x.ima <- ses(x)
summary(x.ima)
```

```
Forecast method: Simple exponential smoothing

Model Information:
Simple exponential smoothing

Call:
ses(y = x)

  Smoothing parameters:
    alpha = 0.1512

  Initial states:
    l = -0.2961

  sigma:  1.0281

     AIC     AICc      BIC
475.7123 475.9597 483.5576

Error measures:
                     ME     RMSE       MAE MPE MAPE      MASE        ACF1
```

```
Training set -0.1262298 1.017906 0.8205515 Inf  Inf 0.7671856 0.003228924

Forecasts:
    Point Forecast     Lo 80      Hi 80     Lo 95       Hi 95
102      -2.224089 -3.541699 -0.9064784 -4.239200 -0.20897761
103      -2.224089 -3.556679 -0.8914981 -4.262110 -0.18606725
104      -2.224089 -3.571493 -0.8766844 -4.284766 -0.16341158
105      -2.224089 -3.586146 -0.8620317 -4.307175 -0.14100230
106      -2.224089 -3.600642 -0.8475351 -4.329346 -0.11883154
107      -2.224089 -3.614988 -0.8331895 -4.351286 -0.09689184
108      -2.224089 -3.629187 -0.8189903 -4.373001 -0.07517613
109      -2.224089 -3.643244 -0.8049332 -4.394500 -0.05367768
110      -2.224089 -3.657164 -0.7910140 -4.415787 -0.03239010
111      -2.224089 -3.670949 -0.7772287 -4.436870 -0.01130731
```

```
x.ima$model
```

```
Simple exponential smoothing

Call:
ses(y = x)

  Smoothing parameters:
    alpha = 0.1512

  Initial states:
    l = -0.2961

  sigma:  1.0281

     AIC     AICc      BIC
475.7123 475.9597 483.5576
```

```
plot(x.ima, main = "", las = 1)
lines(fitted(x.ima), col = "red")
```

$\hat{\alpha} = 0.15, \hat{\sigma} = 1.03$. Note the flat forecast in Figure 5. The two shades of blue are 80 and 95% prediction intervals, respectively, for 10 steps ahead. In the above example the true value of $\alpha = 1 - \lambda = 0.2$.

Simple exponential smoothing as a forecasting method is optimal for other models besides the ARIMA(0, 1, 1) model, including the random-walk plus noise state-space model (Chatfield et al. 2001).

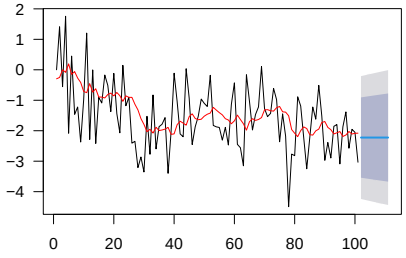


Figure 5: Simple exponential smoothing applied to simulated ARIMA(0, 1, 1) time series, fitted values in red.

`ses` does not seem to return SEs for estimates. One could get SEs from the Hessian matrix when using `optim` directly. Then we would be able to check if the estimates are reasonably close to the simulated values. `ses` returns `states`, which refer to the level estimates, and `fitted`, which refer to one-step-ahead forecasts.

2.4 Example SA Exports

The data set `global_economy` in package `tsibbledata` contains economic indicators featured by the World Bank (1960-2017). We will look at South African exports (% of GDP).

```
library(fpp3)

SAeconomy <- global_economy %>%
  filter(Country == "South Africa")
SAeconomy %>%
  autoplot(Exports) +
  ylab("Exports (% of GDP)") + xlab("Year")
```

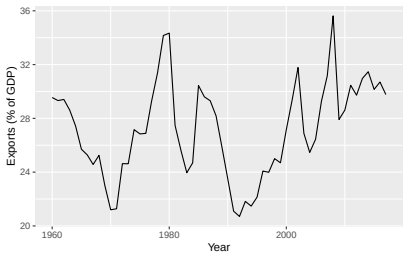


Figure 6: South African exports. Source: R package tsibbledata.

The smoothing parameter α is estimated as 0.9999, the initial state $\ell_0 = 29.55$ and $\hat{\sigma} = 2.3403$. This means that the forecast is almost entirely dominated by the current observation. This works reasonably well in periods with slowly changing mean, but not very well where the mean changes abruptly.

```
library(forecast)

exports <- ts(SAeconomy$Exports, start = 1960)

fit.ses <- ses(exports, data = SAeconomy)
summary(fit.ses)
```

```
Forecast method: Simple exponential smoothing

Model Information:
Simple exponential smoothing

Call:
ses(y = exports, data = SAeconomy)

  Smoothing parameters:
    alpha = 0.9999

  Initial states:
    l = 29.55
```

```
  sigma:  2.3403

     AIC     AICc      BIC
338.1024 338.5468 344.2837

Error measures:
                      ME     RMSE      MAE        MPE     MAPE      MASE
Training set 0.003844136 2.299588 1.635474 -0.3177893 5.978172 0.9827749
                   ACF1
Training set 0.07426042

Forecasts:
     Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
2018       29.77298 26.77378 32.77219 25.18609 34.35987
2019       29.77298 25.53168 34.01428 23.28647 36.25949
2020       29.77298 24.57855 34.96741 21.82879 37.71717
2021       29.77298 23.77502 35.77094 20.59990 38.94607
2022       29.77298 23.06709 36.47887 19.51721 40.02875
2023       29.77298 22.42707 37.11889 18.53839 41.00758
2024       29.77298 21.83851 37.70745 17.63826 41.90770
2025       29.77298 21.29069 38.25527 16.80044 42.74552
2026       29.77298 20.77617 38.76979 16.01355 43.53242
2027       29.77298 20.28952 39.25644 15.26928 44.27669
```

```r
plot(fit.ses, las = 1, main = "",
     ylab = "SA Exports (% of GDP)")
lines(fitted(fit.ses), col = "red")
```

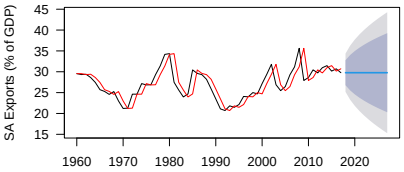


Figure 7: Simple exponential smoothing applied to South African exports data with 10-year forecast (blue), one-step-ahead forecasts (red), 80 and 95% prediction intervals (shaded areas).

The large α value indicates that forecasting SA exports is difficult, there is not much information in the past observations. We might as well use a naive forecast: $\alpha = 1$.

In addition to the parameter estimates, R will calculate a range of prediction accuracy measures (Hyndman & Athanasopoulos 2021, Chapter 5; Hyndman & Koehler 2006): ME (Mean Error), RMSE (Root Mean Squared Error), MAE (Mean Absolute Error), MPE (Mean Percentage Error), MAPE (Mean Absolute Percentage Error), MASE (Mean Absolute Scaled Error), ACF1 (Autocorrelation of errors at lag 1).

3 Forecasting

Simple exponential smoothing is a straightforward procedure for creating point forecasts. However, with a model one can do model checking, and obtain uncertainty estimates for prediction intervals. Given a time series model, how do we construct a forecast, e.g. one-step-ahead forecast? For example, consider the model

$$y_t = y_{t-1} + e_t + \lambda e_{t-1}$$

where e_t is independent noise with mean 0 and variance σ^2. Given observations $y_1, ...y_t$, how should we forecast y_{t+1}?

In the ARIMA framework, what kind of model is this?

Forecasts from a model are created as follows (Hyndman & Athanasopoulos 2021, Section 9.8; Chatfield 2004, pg. 83). The minimum mean squared error forecast of X_{t+h} at time t is the conditional expectation of X_{t+h} at time t: $\hat{X}_{t+h} = E(X_{t+h}|X_t, X_{t-1}, \ldots)$.

1. Write the model so that y_t is on the left hand side and all other terms on the right.
2. Rewrite the equation by replacing t with $T+1$ ($T+h$).
3. On the right hand side of the equation, replace future observations with their forecasts (conditional expectation), future errors with zero, and present and past errors and observations with their observed values (residuals) and observations.

Then, for the above model we obtain

$$\begin{aligned} y_{t+1} &= y_t + e_{t+1} + \lambda e_t && (1) \\ &= y_t + 0 + \lambda e_t && (2) \end{aligned}$$

Show that this is equivalent to a simple exponential smoothing forecast. See also Exercises.

4 Prediction Intervals

For constructing prediction intervals we need an estimate of the prediction error variance. ARMA models can be rewritten as

infinite-order MA processes. The resulting ψ weights are useful for calculating forecast error variances.

We can write an ARMA model as

Chatfield pg. 83

$$x_{n+h} = z_{n+h} + \psi_1 z_{n+h-1} + \dots.$$

where z_i is the *innovation* or error term at time i. The expected value of future errors equals 0. Then

$$\begin{aligned}\hat{x}_{n+h} &= \psi_h z_n + \psi_{h+1} z_{n-1} + \dots \\ &= \sum_{j=0}^{\infty} \psi_{h+j} z_{n-j}\end{aligned}$$

and the h-step-ahead forecast error is

$$z_{n+h} + \psi_1 z_{n+h-1} + \ldots + \psi_{h-1} z_{n+1}.$$

The variance of the h-steps-ahead forecast error is

$$(1 + \psi_1^2 + \ldots + \psi_{h-1}^2)\sigma_z^2,$$

where σ_z^2 is the variance of the independent white noise process z_t. The z_t are estimated from the residuals (one-step-ahead forecast errors).

The *forecast error* is defined as the difference between the observed and predicted value: $e_{T+h} = y_{T+h} - \hat{y}_{T+h|T}$.

Residuals are calculated on the training set, forecast errors are calculated on the test set. Residuals are always calculated from one-step-ahead forecasts. Forecast errors can involve multi-step ahead forecasts.

For the case of simple exponential smoothing, we don't have a model. However, we can estimate the one-step-ahead forecast errors using residuals and then (if we are willing to assume normally distributed errors) construct a prediction interval for a one-step-ahead forecast ($h = 1$)as

$$\hat{y}_{T+h|T} \pm 1.96 \times \hat{\sigma}_h.$$

Prediction intervals for $k > 1$ steps ahead are more complicated. Bootstrapping residuals and simulating paths is one approach which does not require normally distributed residuals.

An estimate of the forecast error variance, $\hat{\sigma}_h^2$, needs to account for the number of parameters estimated. In the case of simple exponential smoothing that would be ℓ_0 and α: $\sqrt{\frac{1}{T-2}\sum_{i=1}^{T} e_i^2}$. At least this is what I think, but one of the exploration questions is about checking this in detail.

In a prediction interval we need to capture $Var(Y_{t+h})$, not the variance of the forecasted point estimate.

Prediction intervals of many models and approaches are too narrow because they do not account for all the uncertainties (Hyndman et al. 2002). Some important sources of variability when forecasting are (fpp2 Section 11.4: https://otexts.com/fpp2/bootstrap.html):

1. The random error term;
2. The parameter estimates;
3. The choice of model for the historical data;
4. The continuation of the historical data generating process into the future.

Many approaches only account for the first of these. But see fpp3 Section 12.5 https://otexts.com/fpp3/bootstrap.html for a bagging approach, which can probably account for sources 1-3.

4.1 Bootstrapping for Prediction Intervals

Alternatively, we can simulate future (forecast) paths using bootstrapped residuals. The residuals are obtained as (observed - predicted). These forecast errors should be independent if the model is appropriate, and this bootstrapping approach does assume independent errors, but does not require normally distributed residuals.

$$\hat{e}_t = \hat{y}_t - y_t$$

Then, to obtain a prediction interval at time $t + k$, we can take percentiles of the simulated y_{t+k}.

fpp3 Sections 5.5 and 12.5: https://otexts.com/fpp2/prediction-intervals.html#prediction-intervals https://otexts.com/fpp2/bootstrap.html

```
library(fpp2)
library(astsa)

g.ses <- ses(goog200, h = 10, level = 95)
plot(g.ses, ylab = "$US", las = 1)
g.fitted <- g.ses$fitted
lines(g.fitted, col = "red")
```

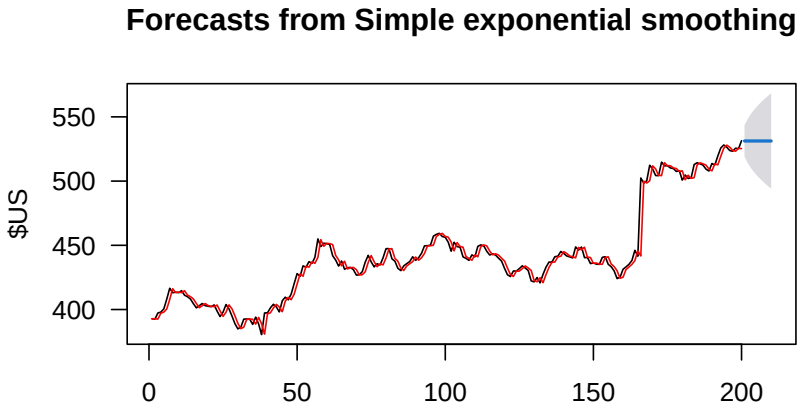


Figure 8: Daily closing stock prices of Google Inc, 25 February 2013 to 13 February 2017, first 200 days. Source: goog200 in fpp2.

```
res <- g.ses$residuals
acf2(res)
```

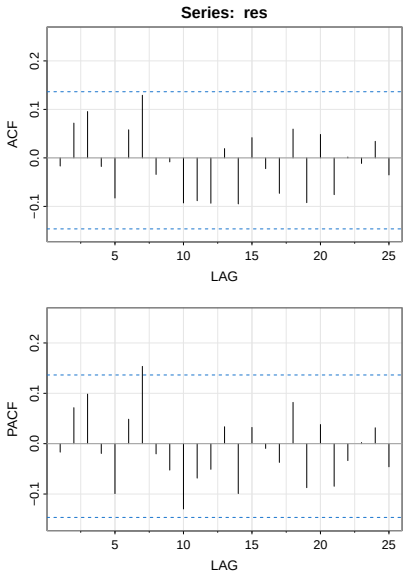


Figure 9: Residual plots from fitting simple exponential smoothing to goog200 data.

There is no apparent dependence in the residuals: it seems reasonable to assume that they are independent.

In the following code chunk we simulate paths by adding bootstrapped residuals (see ETS section at the end of this chapter for the error correction form used here for simulation.) Simulated paths (B = 5000) are kept in rows. 10 columns refer to the simulated observations for $h = 1, \ldots, 10$.

```
alpha <- g.ses$model$fit$par[1]
B <- 5000

yfut <- matrix(0, nrow = B, ncol = 10)     # observation
lev <- matrix(0, nrow = B, ncol = 10)      # level
err <- matrix(0, nrow = B, ncol = 10)      # error

lev200 <- alpha * goog200[200] +
  (1 - alpha) * g.fitted[200]
err[, 1] <- sample(res, size = B, replace = TRUE)
lev[, 1] <- lev200 + alpha * err[, 1]
yfut[, 1] <- lev200 + err[, 1]

for (h in 2:10) {
  err[, h] <- sample(res, size = B, replace = TRUE)
  lev[, h] <- lev[, h-1] + alpha * err[, h]
```

Simulating from Exponential Smoothing: For this we can use the state-space model form of exponential smoothing (see Section 8). The exponential smoothing equations can be rewritten as:

$$y_t = \ell_{t-1} + e_t$$

$$\ell_t = \ell_{t-1} + \alpha e_t.$$

To simulate, we bootstrap the errors. Note that this does not require normally distributed errors, only independent errors.

```
  yfut[, h] <- lev[, h-1] + err[, h]
}

plot(160:200, goog200[160:200], type = "l", xlab = "",
     xlim = c(160, 215), ylim = c(400, 600),
     ylab = "Google closing stock price", las = 1)

for (i in 1:30) {
  lines(201:210, yfut[i,], col = i)
}
```

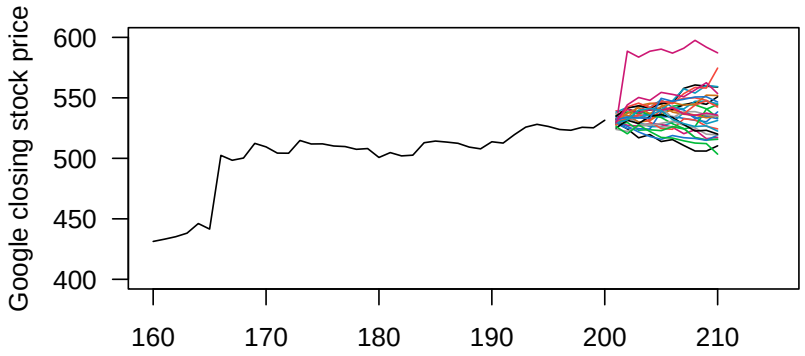


Figure 10: 30 simulated paths to forecast google200 stock prices. The paths are simulated using a state-space model representation of exponential smoothing and bootstrapped residuals.

```
g.bpi <- apply(yfut, 2, quantile,
               prob = c(0.025, 0.975))
fore.bpi <- forecast(g.ses, h = 10, boostrap = TRUE)
pis <- cbind(g.ses$lower, g.ses$upper,
             g.bpi[1,], g.bpi[2,],
             fore.bpi$lower, fore.bpi$upper)

pi.df <- matrix(NA, nrow = 210, ncol = 7)
pi.df[150:200, 1 ] <- goog200[150:200]
pi.df[201:210, 1] <- lev200
pi.df[201:210, 2:7] <- pis
pi.df <- as.data.frame(pi.df)
names(pi.df) <- c("y", "lower", "upper", "b.lower", "b.upper",
                  "lower3", "upper3")

ggplot(data = pi.df[150:210, ], aes(x = 150:210, y = y)) +
         geom_line() +
         geom_ribbon(aes(ymin = lower, ymax = upper),
```

```
                    linetype=2, alpha=0.1, col = "red") +
        geom_ribbon(aes(ymin = b.lower, ymax = b.upper),
                    linetype=2, alpha=0.1, col = "blue") +
        geom_ribbon(aes(ymin = lower3, ymax = upper3),
                    linetype=2, alpha=0.1, col = "green") +
  labs(x="", y="$US")
```

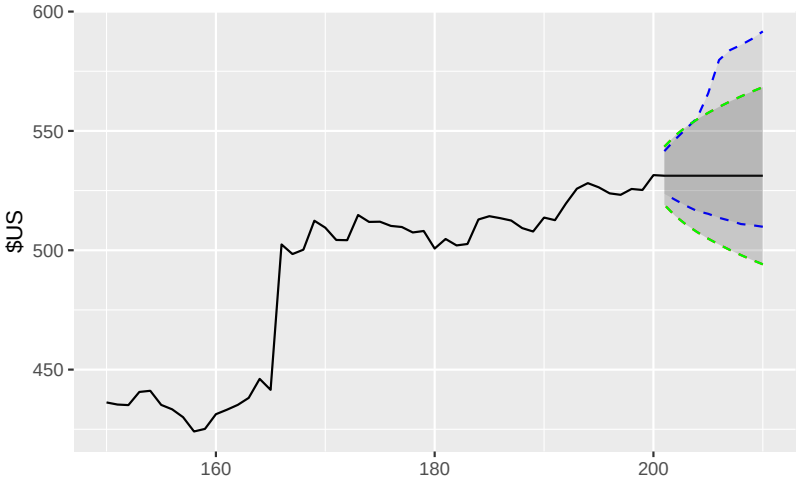


Figure 11: Three different 95% prediction intervals for the next 10 days for Google200 stock prices. Default from forecast uses normal prediction intervals (red). forecast with bootrapping does block bootstrapping of sections and reassembles them, and then simulates paths (I think) (green). Simulated paths using bootstrapped residuals (blue). The red and green predicition intervals are equivalent.

4.1.1 Example 1 with a Bayesian approach

Back to Example 1, where we simulated data from a simple exponential smoothing process. Here is a comparison of the prediction intervals from `forecast` using the `ses` function, and a Bayesian prediction interval. Note that the Bayesian prediction interval is wider because it accounts for the uncertainty in the parameter estimates. Both contain the realised time series from time points 51 to 100.

The Stan model is shown below. The model was fitted using `cmdstanr`. Note the predictions in the *generated quantities* block.

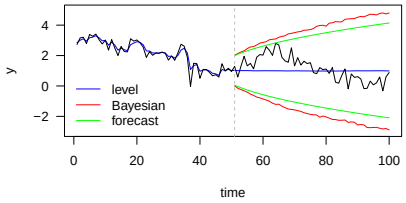


Figure 12: Comparison of Bayesian and `forecast` prediction intervals for Example 1.

```
// Simple Exponential Smoothing
// https://groups.google.com/g/stan-users/c/zAL8ViIKypk
// return values derived from variables defined in model block

data {
```

```
  int<lower=0> N;
  vector[N] y;
  int<lower=0> N_new;
}

parameters {
  real alpha;
  real l0;
  real<lower=0> sigma;
}

model {
  vector[N] level;
  vector[N] err;

  y[1] ~ normal(l0, sigma);
  level[1] = l0 + alpha * (y[1] - l0);

  for (h in 2:N) {
    y[h] ~ normal(level[h-1], sigma);
    level[h] = level[h-1] + alpha * (y[h] - level[h-1]);
  }
}

generated quantities {
  vector[N + N_new] level;
  vector[N_new] y_new;

  level[1] = alpha * y[1] + (1 - alpha) * l0;

  for (t in 2:N)
    level[t] = alpha * y[t] + (1 - alpha) * level[t-1];

  y_new[1] = normal_rng(level[N], sigma);
  level[N + 1] = level[N] + alpha * (y_new[1] - level[N]);

  for (h in 1:N_new) {
    y_new[h] = normal_rng(level[N + h - 1], sigma);
    level[N + h] = level[N + h - 1] + alpha * (y_new[h] - level[N + h-1]);
  }
}
```

5 Holt's Exponential Smoothing for (Linear) Trend

The simple exponential smoothing (exponentially weighted moving average) approach for forecasting was extended by Holt for data with trend, to Holt's (two-parameter) exponential smoothing (also called **double exponential smoothing**). In this approach the trend term (as well as the level term) is updated using exponential smoothing.

$$\begin{aligned}
\text{Forecast equation} \quad & \hat{y}_{t+h|t} = \ell_t + hb_t \\
\text{Level equation} \quad & \ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}) \\
\text{Trend equation} \quad & b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1},
\end{aligned}$$

b_t is the current estimate of increase per unit time: trend or slope. As before, ℓ_t is calculated as a weighted average between the predicted level and the observation at time t. To obtain the predicted level at time t we need to add the predicted change in level (mean) to the previous level ℓ_{t-1}. Once level has been updated, trend is updated as a weighted average of observed and previous trend estimate. Finally, the forecast incorporates the updated level and trend estimates. There are two smoothing parameters. The forecasts are current level plus h times current slope estimate. This gives rise to a linear trend forecast.

5.1 Example: SA Population

As an example we look at the South African population. There clearly is a trend.

```
library(dplyr)
sa.pop <- global_economy %>%
  filter(Country == "South Africa") %>%
  mutate(Pop = Population / 1e6)
#sa.pop

pop <- ts(sa.pop$Pop, start = 1960)
plot(pop, ylab = "SA population (millions)", las = 1,
     xlab = "")
```

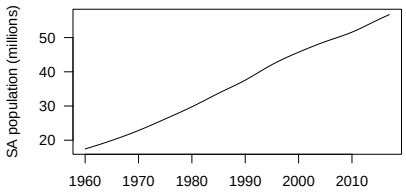


Figure 13: South African population, in millions, 1960-2017. Source: global_economy in package tsibbledata.

```
pop.holt <- holt(pop)
summary(pop.holt)

plot(pop.holt, main = "", las = 1, ylab = "SA population (millions)")
lines(fitted(pop.holt), col = "red")
```

Estimates for the smoothing parameters are: $\alpha = 0.9999$, $\beta = 0.9999$. This puts almost all the weight on the current observation, and on the current change in level. The fitted values (one-step-ahead forecasts) are visually indistinguishable from the observations, which indicates that the one-step-ahead predictions here work really well, but on the other hand, are almost entirely based on current level and trend estimates.

5.2 Damped Trend

Holt's linear trend method produces forecasts which change linearly. In reality trends tend to behave less extreme. One can dampen the effect of the linear trend by introducing a damping parameter ϕ which has the effect of flattening the slope, more for forecasts further into the future, until it is flat for distant forecasts.

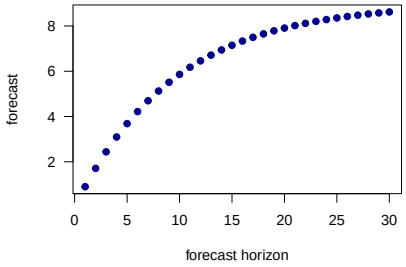


Figure 14: Damped Forecast, with $\phi = 0.9$, slope $= 1$.

$$\begin{aligned}\hat{y}_{t+h|t} &= \ell_t + (\phi + \phi^2 + \cdots + \phi^h)b_t \\ \ell_t &= \alpha y_t + (1-\alpha)(\ell_{t-1} + \phi b_{t-1}) \\ b_t &= \beta(\ell_t - \ell_{t-1}) + (1-\beta)\phi b_{t-1}.\end{aligned}$$

Wherever trend is used to predict ahead, its effect gets dampened.

```
pop.holt2 <- holt(pop, damped = TRUE)

plot(pop.holt2, las = 1, ylab = "SA population (millions)")
lines(fitted(pop.holt2), col = "red")
```

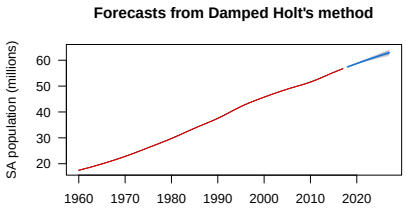


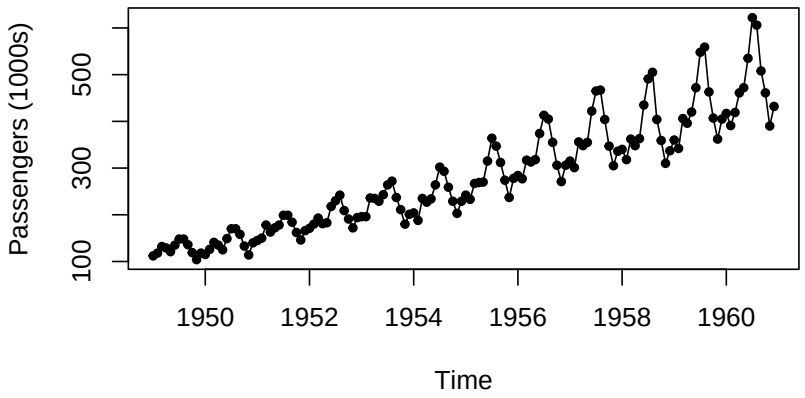
Figure 15: SA Population forecast with Holt's method with damped trend.

5.3 Example Airline Passengers

The classic airline data contains monthly totals of international airline passengers from 1949 to 1960 (data source: `AirPassengers` in library `datasets`.)

```
library(astsa)
library(fpp2)
library(forecast)

data("AirPassengers")
plot(AirPassengers, ylab="Passengers (1000s)", type="o", pch =20)
```




```
ap.fc <- holt(AirPassengers)

plot(ap.fc, main = "", las = 1, ylab = "Air Passengers (thousands)", xlab = "year")
lines(fitted(ap.fc), col = "red")
```

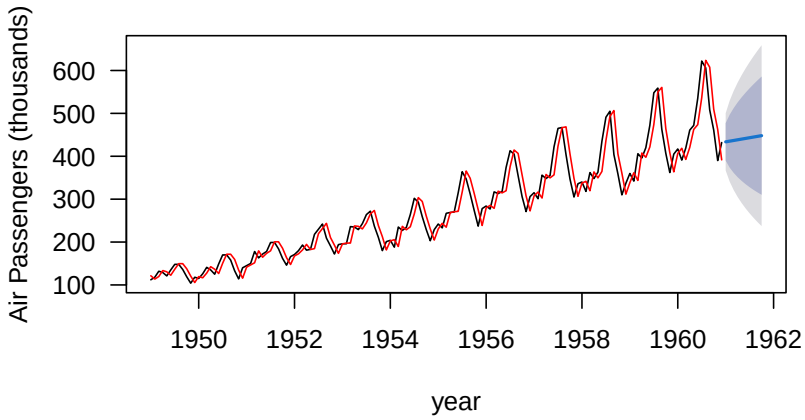


Figure 16: Holt's exponential smoothing applied to the air passenger data

Try answering these questions before reading on. The answers are given below. Consider Figure 16.

1. Do the one-step-ahead predictions work well?
2. Is Holt's exponential smoothing a good prediction method for these data?
3. Can you roughly guess the values of the smoothing parameters α and β?

```
ap.holt <- holt(y = AirPassengers)
ap.holt$model
```

```
Holt's method

Call:
holt(y = AirPassengers)

  Smoothing parameters:
    alpha = 0.9999
    beta  = 1e-04

  Initial states:
    l = 119.7517
    b = 1.5963

  sigma:  34.0125

     AIC     AICc      BIC
1737.295 1737.729 1752.144
```

The main problem with the above forecasts is that they did not consider the clear seasonality present in the data. If seasonality is ignored, the level needs to account for all of this seasonality, will need to change rapidly and therefore needs to adapt to the current observation. This is reflected in a large α estimate of 0.9999. On the other hand, the slowly changing increase in the mean was picked up, and here we cannot rely on the current estimate of slope because that again is dominated by seasonality (with frequent changes in slope), but need to take a long-term average, and therefore would expect a very small β estimate ($\hat{\beta} = 0.0004$). The forecast of trend is maybe not bad, but the forecasts don't contain seasonal effects.

6 Holt-Winters Forecasts for Data with Seasonality

Holt-Winters (triple) exponential smoothing includes a seasonal term, in addition to level and trend. There are three smoothing (updating) equations.

$$
\begin{aligned}
\hat{y}_{t+h|t} &= \ell_t + hb_t + s_{t+h-m} \\
\ell_t &= \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}) \\
b_t &= \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1} \\
s_t &= \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}
\end{aligned}
$$

ℓ_t, b_t, s_t are the local level, trend and seasonal effect at time t. m here denotes the number of seasons in a year (frequency). The m seasonal effects are deviations from the mean (level), and will add up to (approximately) 0. Each observation is made up as: $y_t = \ell_t + s_t + e_t$.

To obtain the current 'observed' level we subtract the seasonal effect from the observation first: $y_t - s_{t-m}$, where s_{t-m} is the current estimate of the current season effect. The trend equation is the same as in the Holt equations. Finally, for the seasonal effect update, we remove the latest level estimate from the observation for the 'observed seasonal effect', and combine this with the predicted seasonal effect, which is the previous estimate for the same season.

This is similar to an ANOVA model with effects parametrization where the group means deviate around an overall mean and the effects add up to zero.

The forecast is constructed as 'level + trend + season', where trend is the expected change in level in h steps.

For all the exponential smoothing updating equations there are alternative error-correction forms. They are equivalent to the form used here, however, we will not cover the error-correction equations here.

6.1 Example: Air Passengers

Here we apply the Holt-Winters method to the Air Passenger data.

```
ap.hw <- hw(AirPassengers)
ap.hw$model
```

```
Holt-Winters' additive method

Call:
hw(y = AirPassengers)

  Smoothing parameters:
    alpha = 0.9935
    beta  = 2e-04
    gamma = 6e-04

  Initial states:
    l = 120.9608
    b = 1.3934
    s = -29.1816 -54.3842 -20.7169 15.0727 65.1554 66.1846
           33.5822 -4.232 -8.0946 -3.8205 -34.3364 -25.2288

  sigma:  18.0471

     AIC     AICc      BIC
1565.872 1570.729 1616.359
```

```
round(forecast::accuracy(ap.hw),2)
```

```
                ME  RMSE   MAE  MPE MAPE MASE ACF1
Training set 0.96 17.01 12.81 0.35 5.22  0.4 0.19
```

```
plot(ap.hw, las = 1, ylab = "air passengers (thousands)", main = "")
lines(fitted(ap.hw), col = "red")
```

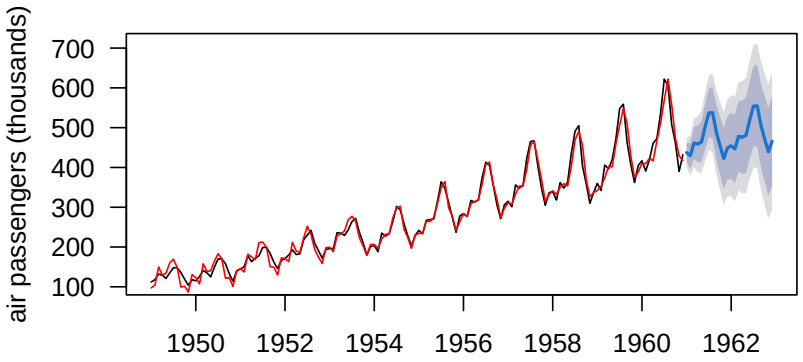


Figure 17: Holt-Winters exponential smoothing applied to the air passenger data. The red line adds fitted values.

Estimates for the smoothing parameters are: $\alpha = 0.9935, \beta = 0.0002, \gamma = -.0006$. Both seasonal and trend smoothing parameters are very small, indicating that these updates are formed as long-term averages. This explains why the peaks in the forecasts are much smaller than the last few observed peaks. The updated level on the other hand depends mostly on the latest observation. The method is not able to pick up the increases (larger variation) in the seasonal effects.

7 Additive vs Multiplicative

In the three methods we have covered so far the observations were assumed to be a sum (additive) of components level, season and error. Many processes are additive, however there are also naturally multiplicative processes. Processes which change as percentages or rates are multiplicative. This is common for counts, e.g. passenger counts, internet traffic, or populations: for every person we have this year, we will have λ next year. In the air passenger data we can see that the seasonal variations are proportional to level. To establish whether our process is multiplicative or additive, we need to ask a question such as: will the effect increase by a constant R20 every year or by 5% every year?

If the seasonal effect is multiplicative, the size of the seasonal effect depends on the level. This is similar to an interaction, and therefore some sources refer to this as an interaction between level and seasonality. A log-transformation can be used to make

Level is often referred to as 'trend'. In these notes we prefer to use 'trend' for 'change in level'.

the seasonal effect additive. Alternatively, the updating equations in exponential smoothing can become multiplicative:

$$\begin{aligned}
\hat{y}_{t+h|t} &= (\ell_t + hb_t)s_{t+h-m} \\
\ell_t &= \alpha(y_t/s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}) \\
b_t &= \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1} \\
s_t &= \gamma(y_t/\ell_t) + (1-\gamma)s_{t-m}
\end{aligned}$$

In addition to multiplicative seasonal effects, we can also have multiplicative errors. This leads to three types of seasonal models:

$$\begin{aligned}
Y_t &= \ell_t + s_t + e_t \\
Y_t &= \ell_t s_t + e_t \\
Y_t &= \ell_t s_t e_t
\end{aligned}$$

Multiplicative errors can be picked up from residual plots: they will show increased variance with increasing fitted value (level) if we should have used a multiplicative model but did use an additive model.

```
plot(c(residuals(ap.hw)) ~ c(fitted(ap.hw)), las = 1,
     pch = 19, type = "p",
     xlab = "fitted values", ylab = "residuals")
abline(h = 0, col = "grey")
```

The variance for larger fitted values does seem a bit larger.

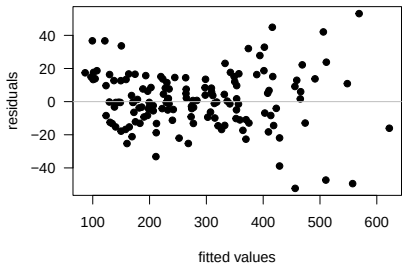


Figure 18: Residuals for airline passenger data after fitting Holt-Winters model.

7.1 Example: Air Passengers

What happens if we allow `ets()` to choose an appropriate model for the air passenger data?

```
fit.a <- ets(AirPassengers)
fit.a
```

`ets` (package `forecast`) fits exponential smoothing models using a state-space methodology. It automatically chooses a best model from the range of possible types of exponential smoothing models (additive or multiplicative, damped or not, with or without season and trend).

```
ETS(M,Ad,M)

Call:
ets(y = AirPassengers)

  Smoothing parameters:
    alpha = 0.7096
    beta  = 0.0204
    gamma = 1e-04
    phi   = 0.98

  Initial states:
    l = 120.9939
    b = 1.7705
    s = 0.8944 0.7993 0.9217 1.0592 1.2203 1.2318
           1.1105 0.9786 0.9804 1.011 0.8869 0.9059

  sigma:  0.0392

     AIC     AICc      BIC
1395.166 1400.638 1448.623
```

```
plot(forecast.ets(fit.a), main = "", las = 1,
     ylab = "air passengers")
lines(fitted(fit.a), col = "red")
```

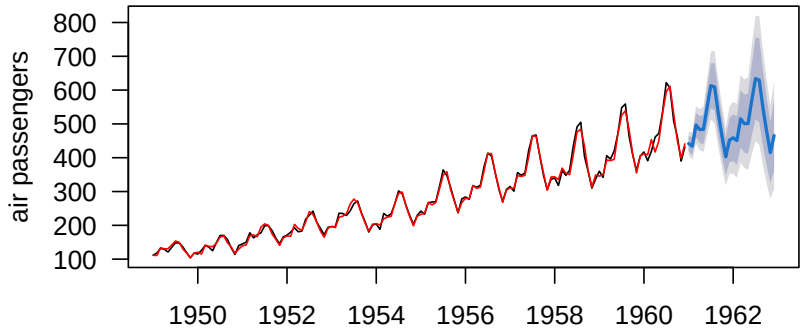


Figure 19: Automatic choice of model for air passenger data: multiplicative error and seasonal effets.

It chooses multiplicative errors and seasonal effects (M, Ad, M) and additive damped (Ad) trend. It knows about the seasonal effects because the data are stored as a time series object.

8 Simple exponential smoothing as a state-space model

Simple exponential smoothing has

$$\begin{aligned} \text{Forecast equation} \quad & \hat{y}_{t+h|t} = \ell_t \\ \text{Smoothing equation} \quad & \ell_t = \alpha y_t + (1-\alpha)\ell_{t-1}. \end{aligned}$$

We can rewrite the smoothing equation as

$$\begin{aligned} \ell_t &= \alpha y_t + (1-\alpha)\ell_{t-1} \\ &= \alpha y_t + (1-\alpha)\hat{y}_t \\ &= \hat{y}_t + \alpha(y_t - \hat{y}_t) \\ &= \ell_{t-1} + \alpha e_t \end{aligned}$$

This is also known as the **error correction form** of simple exponential smoothing: the current level estimate is adjusted according to the error (difference between observation and previous level estimate). This form is used to simulate paths for simulated prediction intervals. It can also be written as $\hat{y}_{t+1} = \hat{y}_t + \alpha e_t$.

and

$$y_t = \ell_{t-1} + e_t.$$

If we further assume a distribution for the errors e_t (e.g. $e_t \sim N(0, \sigma^2)$), we have a state equation and an observation equation, which together form a state-space model. The state here refers to the underlying, unobserved level, which we would like to estimate based on our observations y_t.

$$\begin{aligned} y_t &= \ell_{t-1} + e_t \\ \ell_t &= \ell_{t-1} + \alpha e_t \end{aligned}$$

With this model we can estimate σ, useful for constructing prediction intervals.

9 ETS: error, trend, seasonality

The `ets()` function in the `forecast` R library will automatically attempt to choose an appropriate forecasting equation from the possible combinations of additive, multiplicative, seasonal or not, damped or not models (Hyndman & Athanasopoulos 2021, Section 8.4), 12 possible models.

Exponential smoothing algorithms are closely related to state-space models, with the exception that exponential smoothing does not involve a stochastic part, or a model. Have another look at the exponential smoothing equations. The first equation defines the observation or measurement, and can therefore be called a measurement or observation equation. The other equations define how parameters, or state variables, change over time. If we describe the uncertainty (error terms) using statistical models, we have a (special case of a) state-space model. The advantage of having an actual model is that this allows us to obtain a predictive distribution, not just a point forecast.

The `ets()` function uses state-space models to fit and forecast. More on state-space models later.

10 Exercises

10.1

Show that the forecast variance for an ETS(A,N,N) model is given by $\sigma^2 \left[1 + \alpha^2(h-1)\right]$.

10.2

Write down a 95% prediction interval for an ETS(A,N,N) model as a function of ℓ_T, α, h, and σ assuming normally distributed errors.

10.3

Consider the ARIMA(0, 1, 1) process

$$(1 - B)X_t = (1 - \theta B)Z_t$$

Show that $\hat{x}_{t+1|t} = x_t - \theta z_t$, and $\hat{x}_{t+h|t} = \hat{x}_{t+h-1|t}$ for $h \geq 2$.

Express $\hat{x}_{t+1}$ in terms of x_t and $\hat{x}_{t|t-1}$ and show that this is equivalent to exponential smoothing.

By considering the ψ weights of the process, show that the variance of the h-steps-ahead prediction error is

$$[1 + (h - 1)(1 - \theta)^2]\sigma_z^2$$

11 References

- Chatfield. 2004. The Analysis of Time Series. An Introduction. 6th ed. Chapman & Hall.
- Chatfield C, Koehler AB, Ord JK and Snyder RD. 2001. A new look at models for exponential smoothing. The Statistician 50: 147-159.
- Hyndman RJ & Athanasopoulos. Forecasting: Principles and Practice. https://otexts.com/fpp2/. (exponential smoothing)
- Hyndman, R.J., & Athanasopoulos, G. (2021). Forecasting: principles and practice, 3rd edition, OTexts: Melbourne, Australia. https://otexts.com/fpp3/.
- Hyndman RJ & Koehler AB. 2006. Another look at measures of forecast accuracy. International Journal of Forecasting 22(4), 679-688. https://robjhyndman.com/publications/another-look-at-measures-of-forecast-accuracy/
- Hyndman RJ, Koehler AB, Snyder RD, and Grose S. (2002). A state space framework for automatic forecasting using exponential smoothing methods. International J. Forecasting 18(3):439–454. https://doi.org/10.1016/S0169-2070(01)00110-8
- Hyndman, R.J., Koehler, A.B., Ord, J.K., Snyder, R.D. (2008). Forecasting with exponential smoothing: the state space approach. Springer. https://robjhyndman.com/expsmooth/.
- Shumway & Stoffer Section 3.7 Example 3.37
- Statistical and Machine Learning forecasting methods: Concerns and ways forward. Makridakis, Spiliotis & Assimakopoulos. Plos ONE 2018. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0194889 (interesting overview of modern forecasting approaches)